

Assignment 3

GenAIs used: Copilot and Claude both accessed on computer

Prompt:

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next

- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Copilot:

```
#include <iostream>

#include <vector>

#include <string>

#include <sstream>

using namespace std;

// -----
// Email Class
// -----

class Email {
private:
    string sender;
    string subject;
    string date; // MM-DD-YYYY

    int priorityScore; // computed based on sender + date

    int convertDateToInt(string d) {
        // MM-DD-YYYY → YYYYMMDD (integer)
        int mm = stoi(d.substr(0, 2));
        int dd = stoi(d.substr(3, 2));
        int yyyy = stoi(d.substr(6, 4));
        return yyyy * 10000 + mm * 100 + dd;
    }

    int senderPriority(string s) {
```

```

    if (s == "Boss") return 5;
    if (s == "Subordinate") return 4;
    if (s == "Peer") return 3;
    if (s == "ImportantPerson") return 2;
    return 1; // OtherPerson
}

```

public:

```

Email(string s, string subj, string d) {
    sender = s;
    subject = subj;
    date = d;

    int sp = senderPriority(s);
    int dp = convertDateToInt(d);

    // Combine sender priority + date priority
    // Newer dates should be higher priority
    priorityScore = sp * 100000000 + dp;
}

int getPriority() const { return priorityScore; }
string getSender() const { return sender; }
string getSubject() const { return subject; }
string getDate() const { return date; }
};

```

```

// -----
// MaxHeap Class (list-based)
// -----

class MaxHeap {
private:
    vector<Email> heap;

    int parent(int i) { return (i - 1) / 2; }
    int left(int i) { return 2 * i + 1; }
    int right(int i) { return 2 * i + 2; }

    void swapEmails(int i, int j) {
        Email temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    void heapifyUp(int index) {
        while (index != 0 && heap[parent(index)].getPriority() < heap[index].getPriority()) {
            swapEmails(index, parent(index));
            index = parent(index);
        }
    }

    void heapifyDown(int index) {
        int largest = index;
        int l = left(index);

```

```
int r = right(index);
```

```
if (l < heap.size() && heap[l].getPriority() > heap[largest].getPriority())
```

```
    largest = l;
```

```
if (r < heap.size() && heap[r].getPriority() > heap[largest].getPriority())
```

```
    largest = r;
```

```
if (largest != index) {
```

```
    swapEmails(index, largest);
```

```
    heapifyDown(largest);
```

```
}
```

```
}
```

```
public:
```

```
void insert(const Email& e) {
```

```
    heap.push_back(e);
```

```
    heapifyUp(heap.size() - 1);
```

```
}
```

```
bool empty() const {
```

```
    return heap.empty();
```

```
}
```

```
Email* peek() {
```

```
    if (heap.empty()) return nullptr;
```

```
    return &heap[0];
```

```
}
```

```
void removeMax() {  
    if (heap.empty()) return;  
  
    heap[0] = heap.back();  
    heap.pop_back();  
    if (!heap.empty())  
        heapifyDown(0);  
}
```

```
int count() const {  
    return heap.size();  
}  
};
```

```
// -----  
// CEOInbox Class (handles commands)  
// -----
```

```
class CEOInbox {  
private:  
    MaxHeap inbox;  
  
public:  
    void addEmail(string sender, string subject, string date) {  
        Email e(sender, subject, date);  
        inbox.insert(e);  
    }  
};
```

```
}
```

```
void nextEmail() {
```

```
    Email* e = inbox.peek();
```

```
    if (!e) {
```

```
        cout << "No emails in inbox\n";
```

```
        return;
```

```
    }
```

```
    cout << "Sender: " << e->getSender() << "\n";
```

```
    cout << "Subject: " << e->getSubject() << "\n";
```

```
    cout << "Date: " << e->getDate() << "\n";
```

```
}
```

```
void readEmail() {
```

```
    if (inbox.empty()) {
```

```
        cout << "No emails to read\n";
```

```
        return;
```

```
    }
```

```
    inbox.removeMax();
```

```
}
```

```
void countEmails() {
```

```
    cout << inbox.count() << "\n";
```

```
}
```

```
};
```

```
// -----  
// Main Program  
// -----  
  
int main() {  
    CEOInbox system;  
  
    string line;  
    while (getline(cin, line)) {  
        if (line.rfind("EMAIL", 0) == 0) {  
            // EMAIL <sender>, <subject>, <date>  
            string rest = line.substr(6);  
            string sender, subject, date;  
  
            stringstream ss(rest);  
            getline(ss, sender, ',');  
            getline(ss, subject, ',');  
            getline(ss, date);  
  
            system.addEmail(sender, subject, date);  
        }  
        else if (line == "NEXT") {  
            system.nextEmail();  
        }  
        else if (line == "READ") {  
            system.readEmail();  
        }  
        else if (line == "COUNT") {
```

```

        system.countEmails();
    }
}

return 0;
}

```

Claude:

```

// ceo_email_priority.cpp
//
// Prioritizes a CEO's incoming emails using a MaxHeap-based priority queue.
// The MaxHeap is implemented from scratch on top of a std::vector (a
// "list-based" / array-based implementation) -- no STL priority_queue or
// heap algorithms are used.
//
// Commands (read from a file, one per line):
//  EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
//  NEXT
//  READ
//  COUNT
//
// Sender categories, highest priority first:
//  Boss > Subordinate > Peer > ImportantPerson > OtherPerson
// Ties within the same category are broken by date, newest first.
//
// Usage:
//  ceo_email_priority.exe testfile.txt
//  (or pipe/redirect a file into stdin: ceo_email_priority.exe < testfile.txt)

```

```

#include <iostream>

#include <fstream>

#include <sstream>

#include <string>

#include <vector>

#include <stdexcept>


// -----
// Email: represents a single email object with everything needed to
// compare it against other emails for priority purposes.
// -----

class Email {
public:
    Email(const std::string& senderCategory,
          const std::string& subject,
          const std::string& date)
        : senderCategory_(senderCategory),
          subject_(subject),
          date_(date),
          priorityRank_(categoryToRank(senderCategory)),
          dateValue_(parseDate(date))
    {
    }

    const std::string& getSenderCategory() const { return senderCategory_; }
    const std::string& getSubject()      const { return subject_; }

```

```

const std::string& getDate() const { return date_; }

int getPriorityRank() const { return priorityRank_; }

long getDateValue() const { return dateValue_; }

// Returns true if this email should be read before 'other'.
// Higher priorityRank_ wins; ties broken by newer date (larger dateValue_).
bool isHigherPriorityThan(const Email& other) const {
    if (priorityRank_ != other.priorityRank_) {
        return priorityRank_ > other.priorityRank_;
    }
    return dateValue_ > other.dateValue_;
}

void display() const {
    std::cout << "Sender: " << senderCategory_ << "\n";
    std::cout << "Subject: " << subject_ << "\n";
    std::cout << "Date: " << date_ << "\n";
}

private:
    std::string senderCategory_;
    std::string subject_;
    std::string date_;
    int priorityRank_;
    long dateValue_;

    // Boss=5 (highest) ... OtherPerson=1 (lowest)

```

```

static int categoryToRank(const std::string& category) {
    if (category == "Boss")        return 5;
    if (category == "Subordinate")  return 4;
    if (category == "Peer")        return 3;
    if (category == "ImportantPerson") return 2;
    if (category == "OtherPerson")  return 1;
    return 0; // unknown category -> lowest possible priority
}

```

// Converts MM-DD-YYYY into a comparable integer YYYYMMDD.

```

static long parseDate(const std::string& date) {
    int month = 0, day = 0, year = 0;
    char dash1 = '\0', dash2 = '\0';
    std::istringstream iss(date);
    iss >> month >> dash1 >> day >> dash2 >> year;
    return static_cast<long>(year) * 10000L +
           static_cast<long>(month) * 100L +
           static_cast<long>(day);
}
};

```

```

// -----
// MaxHeap: a from-scratch, array/list-based (std::vector-backed) binary
// max-heap of Email objects. Highest priority email is always at index 0.
// -----
class MaxHeap {
public:

```

```
MaxHeap() {}
```

```
bool isEmpty() const { return data_.empty(); }
```

```
int size() const { return static_cast<int>(data_.size()); }
```

```
void insert(const Email& email) {
```

```
    data_.push_back(email);
```

```
    heapifyUp(size() - 1);
```

```
}
```

```
// Returns the highest-priority email without removing it.
```

```
const Email& peekMax() const {
```

```
    if (isEmpty()) {
```

```
        throw std::runtime_error("peekMax() called on empty heap");
```

```
    }
```

```
    return data_[0];
```

```
}
```

```
// Removes and returns the highest-priority email.
```

```
Email extractMax() {
```

```
    if (isEmpty()) {
```

```
        throw std::runtime_error("extractMax() called on empty heap");
```

```
    }
```

```
    Email maxItem = data_[0];
```

```
    data_[0] = data_[size() - 1];
```

```
    data_.pop_back();
```

```
    if (!isEmpty()) {
```

```
        heapifyDown(0);  
    }  
    return maxItem;  
}
```

private:

```
std::vector<Email> data_;
```

```
int parentOf(int i)    const { return (i - 1) / 2; }
```

```
int leftChildOf(int i) const { return 2 * i + 1; }
```

```
int rightChildOf(int i) const { return 2 * i + 2; }
```

```
void heapifyUp(int index) {
```

```
    while (index > 0) {
```

```
        int p = parentOf(index);
```

```
        if (data_[index].isHigherPriorityThan(data_[p])) {
```

```
            std::swap(data_[index], data_[p]);
```

```
            index = p;
```

```
        } else {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
void heapifyDown(int index) {
```

```
    int n = size();
```

```
    while (true) {
```

```

    int left = leftChildOf(index);

    int right = rightChildOf(index);

    int largest = index;

    if (left < n && data_[left].isHigherPriorityThan(data_[largest])) {
        largest = left;
    }

    if (right < n && data_[right].isHigherPriorityThan(data_[largest])) {
        largest = right;
    }

    if (largest == index) {
        break;
    }

    std::swap(data_[index], data_[largest]);
    index = largest;
}

};

// -----
// CEOInbox: the object the rest of the program interacts with. Wraps a
// MaxHeap and implements the EMAIL / NEXT / READ / COUNT behaviors.
// -----

class CEOInbox {
public:
    void addEmail(const std::string& category,
                 const std::string& subject,

```

```

        const std::string& date) {
    heap_.insert(Email(category, subject, date));
}

// NEXT: display the next email to be read, without removing it.
void showNext() const {
    if (heap_.isEmpty()) {
        std::cout << "No emails in inbox.\n";
        return;
    }
    heap_.peekMax().display();
}

```

```

// READ: CEO has read/dealt with the top email; remove it silently.
void readTopEmail() {
    if (heap_.isEmpty()) {
        return; // nothing to read; nothing to display per spec
    }
    heap_.extractMax();
}

```

```

// COUNT: display current count of unread emails.
void showCount() const {
    std::cout << "Unread emails: " << heap_.size() << "\n";
}

```

private:

```

    MaxHeap heap_;
};

// -----
// Helpers for parsing the test-file commands.
// -----

// Splits "EMAIL Boss,Meeting moved,09-15-2026" (the part after "EMAIL ")
// into category / subject / date. Subject is guaranteed not to contain
// commas, so a simple 3-way comma split works.
static bool parseEmailFields(const std::string& rest,
                             std::string& category,
                             std::string& subject,
                             std::string& date) {
    size_t firstComma = rest.find(',');
    if (firstComma == std::string::npos) return false;
    size_t secondComma = rest.find(',', firstComma + 1);
    if (secondComma == std::string::npos) return false;

    category = rest.substr(0, firstComma);
    subject  = rest.substr(firstComma + 1, secondComma - firstComma - 1);
    date     = rest.substr(secondComma + 1);
    return true;
}

static void processLine(const std::string& line, CEOInbox& inbox) {
    if (line.empty()) return;

```

```

std::istringstream lineStream(line);
std::string command;
lineStream >> command;

if (command == "EMAIL") {
    // Everything after "EMAIL " (preserve spaces inside the subject).
    std::string rest;
    std::getline(lineStream, rest);
    // Remove a single leading space left after extracting "EMAIL".
    if (!rest.empty() && rest[0] == ' ') {
        rest = rest.substr(1);
    }
    std::string category, subject, date;
    if (parseEmailFields(rest, category, subject, date)) {
        inbox.addEmail(category, subject, date);
    }
} else if (command == "NEXT") {
    inbox.showNext();
} else if (command == "READ") {
    inbox.readTopEmail();
} else if (command == "COUNT") {
    inbox.showCount();
}
// Unknown/blank commands are silently ignored.
}

```

```

int main(int argc, char* argv[]) {
    CEOInbox inbox;
    std::string line;

    if (argc >= 2) {
        std::ifstream inFile(argv[1]);
        if (!inFile.is_open()) {
            std::cerr << "Could not open file: " << argv[1] << "\n";
            return 1;
        }
        while (std::getline(inFile, line)) {
            processLine(line, inbox);
        }
    } else {
        // No filename given -- read commands from standard input.
        while (std::getline(std::cin, line)) {
            processLine(line, inbox);
        }
    }

    return 0;
}

```

Analysis:

In terms of correctness Claude ran easily with no issues. Copilot's code, however, didn't run at all and came back with multiple issues. Claude couldn't handle if an incorrect file was inputted. In terms of execution time, if copilot ran it would have a better time. It uses mostly if-else statements with one while loop. Claude uses two while loops. Because while loops must go through every possibility before fully executing, they take longer than if-else statements which just find the first condition that matches its needs and runs the next process immediately. When it

comes to space complexity, copilot is higher. While Claude uses more variables than copilot, Claude uses an iterative call for `heapifydown()` where copilot uses a recursive call, which uses more space than an iterative call. This is because the function must remember its current call to make the next call whereas iterative calls must only remember the current call. Lastly in terms of maintainability Claude is better. It has more comments explaining the code and has more error handling and flexible input options. It also clearly separates each responsibility so that if someone wishes to edit a certain section of code it doesn't require rewriting major parts of the code. It also makes it clear what editing each part effects and an error in one part won't fail the entire code. Copilot mostly executes in main so editing one-part effects how the entire function runs since main is what's running the program. This means future edits are more complicated and require considering multiple parts of the program.

I will edit Claude's code. This is because it already runs very efficiently. It also already runs whereas I'd need to figure out what's wrong with Copilot's code to fix it. It would just be easier to work on something that already runs and runs well. To improve the correctness, I will have the code run and then ask for text file. This way if the user inputs an incorrect file it doesn't crash the program and it can ask for a different input. I will also check if the date entered is a valid date. Such as making sure the month exists and the day is valid. For execution time I will reduce unnecessary copying in `extractMax()`. This will reduce the amount of work the program does as it will spend less time on unnecessary operations. Space complexity is already pretty efficient. For maintainability I'll add more comments fully explaining the program. I will also make it easier to add new categories of email priority to the program. This way if someone wants to add or delete something it's much easier.